DWES · 1. Apuntes completos (Spring Boot + Thymeleaf)

Nov 8, 2026 · @vielsajr

0. Cómo usar estos apuntes

Estos apuntes cubren todo lo que aparece en UT0, UT2 (partes 1 y 2), UT3 y UT4, más lo que hace falta para resolver la Hoja04 (club de coleccionistas). Cada apartado lleva una etiqueta que indica de dónde sale:

Etiqueta	Significado
[A]	Está en los apuntes del profesor (se indica la unidad)
[E]	No está explicado en los apuntes, pero lo exige o lo deduce el ejercicio Hoja04
[C]	Complementario: no aparece en el material, pero lo necesitas para que el código funcione o para entenderlo

Material analizado: UT0 (Java), UT2 parte 1 (Spring Boot y MVC), UT2 parte 2 (Controller + Model + Thymeleaf básico), UT3 (Thymeleaf), UT4 (Spring MVC: servicios, formularios, validación, redirecciones, imágenes) y Hoja04_SpringMVC FINAL. **No hay UT1** entre los PDF: si existe, puede faltar contenido.

Lo más importante para el examen: el ejercicio trabaja con **Spring MVC + Thymeleaf y datos en memoria**. No usa base de datos: la persistencia (JPA) llega "en la siguiente unidad", según el propio enunciado. Por eso JPA está en el apartado 11, marcado [C].

Índice

1. Java básico
2. Programación orientada a objetos
3. Colecciones, genéricos, lambdas, Streams y Optional
4. Aplicaciones web, MVC y Spring Boot
5. Controladores y Model
6. Thymeleaf
7. Capa de servicio e inyección de dependencias
8. Formularios, validación y redirecciones

9. Subida de imágenes y @Value
10. Cómo encaja todo
11. Complementario: JPA y base de datos
12. Erratas y contradicciones de los apuntes

1. Java básico [A – UT0]

Spring Boot es Java: si la sintaxis básica falla, nada compila. Este apartado repasa lo imprescindible.

1.1 Estructura de un programa

Todo programa Java arranca en el método `main`. En Spring Boot, `main` está en la clase principal y solo arranca el servidor.

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hola mundo");
    }
}
```

- `public class Main`: el archivo debe llamarse `Main.java` (mismo nombre que la clase pública).
- Cada instrucción termina en `;` y los bloques van entre `{ }`.
- Comentarios: `// una línea` y `/* varias líneas */`.
- Nombres de variables y métodos en **camelCase** (`precioTotal`); clases en **PascalCase** (`MonedaService`); constantes en MAYÚSCULAS (`MAX_INTENTOS`).

1.2 Variables y tipos

Una variable se declara con su **tipo** y su **nombre**. Java es fuertemente tipado: una variable `int` nunca podrá guardar texto.

Tipo	Ejemplo	Nota
`int`	`25`	Entero
`long`	`123456789L`	Entero grande (la `L` final)
`double`	`19.95`	Decimal
`boolean`	`true` / `false`	

Tipo	Ejemplo	Nota
`char`	`'A'`	Comillas **simples**
`String`	`"Hola"`	Comillas **dobles**; es un objeto, no un primitivo

Primitivos vs. clases envoltorio [A – UT2 p2]: `int`, `double`, `boolean`… son primitivos y **no pueden ser null**. `Integer`, `Double`, `Long`, `Boolean` son su versión objeto y **sí admiten null**. En los modelos de Spring conviene usar los envoltorios (`Integer anio`, `Double valor`, `Long id`): si el usuario deja un campo vacío en el formulario llega `null`, y con un primitivo daría error. [C]

1.3 Operadores

- Aritméticos: `+ - * / %` (`10 % 3` es 1, el resto). Ojo: `10 / 3` entre enteros da 3; `10.0 / 3` da 3,333…
- Comparación: `== != > < >= <=`, devuelven `boolean`.
- Lógicos: `&&` (y), `||` (o), `!` (no).
- Asignación: `= += -= *= /= ++ --`.
- **`=` asigna; `==` compara.**
- **Para comparar `String` usa siempre `.equals()`**, nunca `==` (`==` compara si son el mismo objeto en memoria) [A – UT2 p2]. Lo mismo vale para `Long` e `Integer`: compara ids con `.equals()` [C].
- Ternario: `String estado = (nota >= 5) ? "Aprobado" : "Suspenso";` [A – UT2 p2].

1.4 Condicionales y bucles

```
if (nota < 5) {
    // suspenso
} else if (nota < 7) {
    // aprobado
} else {
    // notable o más
}

// switch moderno con flecha: no necesita break
String nombre = switch (dia) {
    case 1 -> "Lunes";
    case 2 -> "Martes";
    default -> "Desconocido";
};

for (int i = 0; i < 10; i++) { }        // número de vueltas conocido
for (Moneda m : monedas) { }            // for-each: recorre una colección
while (condicion) { }                   // mientras se cumpla
do { } while (condicion);               // al menos una vez
```

`break` sale del bucle y `continue` salta a la siguiente vuelta (los apuntes recomiendan no abusar).

1.5 Entrada/salida y conversión

`System.out.println()` escribe en consola (en Spring sirve para depurar). `Scanner` lee de teclado (`import java.util.Scanner;`), pero en una aplicación web no se usa: los datos llegan por la petición HTTP.

Casting: `int entero = (int) 10.5;` da 10. De `int` a `double` la conversión es automática. [C] De texto a número: `Integer.parseInt("25")`, `Double.parseDouble("2.5")`; en Spring esta conversión la hace el framework solo.

1.6 Métodos

Un método agrupa instrucciones que hacen **una tarea concreta**.

```
//  modificadores  tipo devuelto  nombre    parámetros
    public static  int            sumar    (int a, int b) {
        return a + b;      // return devuelve el valor Y termina el método
    }
```

- **Parámetros**: lo que recibe (`a`, `b`). **Argumentos**: los valores concretos al llamarlo (`sumar(4, 5)`).

- **`void`**: no devuelve nada. No puedes hacer `int r = saludar("Ana");` si `saludar` es `void`.
- **Variables locales**: solo existen dentro del método donde se declaran.
- **Paso por valor**: con primitivos, el método recibe una **copia**; cambiarla no cambia la original. [C] Con objetos se copia la **referencia**: el método puede modificar el objeto original (por eso un servicio puede cambiar una `Moneda` que le pasas).
- **`static`**: pertenece a la clase, se llama sin crear objeto (`Calculadora.sumar(3, 4)`).
- **Sobrecarga**: mismo nombre, distintos parámetros. Cambiar solo el tipo devuelto **no** es sobrecarga válida.

Ejemplo típico del curso, un método que recibe datos y devuelve un resultado:

```
public static boolean esMayorDeEdad(int edad) {
    return edad >= 18;
}
```

2. Programación orientada a objetos [A – UT0]

En Spring todo son clases: el modelo (`Moneda`), los servicios y los controladores. Entender clase, objeto, constructor e interfaz es entender el 80 % del ejercicio.

2.1 Clase, objeto y atributos

Una **clase** es el molde; un **objeto** es una pieza hecha con ese molde. Los **atributos** son los datos que guarda cada objeto.

```
public class Producto {
    private String nombre;   // atributo
    private double precio;
}

Producto p1 = new Producto();   // new crea un objeto
Producto p2 = new Producto();   // otro objeto, con sus propios valores
```

2.2 Constructores y this

El **constructor** inicializa el objeto al crearlo. Se llama igual que la clase y **no tiene tipo de retorno**. `this.nombre` es el atributo; `nombre` a secas es el parámetro.

```
public class Producto {
    private String nombre;
    private double precio;

    public Producto() { }                        // constructor vacío

    public Producto(String nombre, double precio) {   // sobrecarga de
constructores
        this.nombre = nombre;
        this.precio = precio;
    }
}
```

Clave para Spring [C]: si escribes un constructor con parámetros, Java deja de crear el vacío automáticamente. Los formularios de Spring necesitan el **constructor vacío** para crear el objeto antes de rellenarlo con los datos. Con Lombok: `@NoArgsConstructor` + `@AllArgsConstructor`.

2.3 Encapsulación: getters y setters

Los atributos son `private`; se accede a ellos con métodos públicos.

```
public String getNombre() { return nombre; }
public void setNombre(String nombre) { this.nombre = nombre; }
public boolean isActivo() { return activo; }    // con boolean el getter
empieza por "is"

public void setPrecio(double precio) {            // un setter puede validar
    if (precio >= 0) { this.precio = precio; }
}
```

Por qué importa en Spring [C]: Thymeleaf y Spring **no leen los atributos directamente**, llaman a los getters y setters. `${moneda.nombre}` ejecuta `getNombre()` y `th:field="*{nombre}"` usa `setNombre()` al enviar el formulario. Si falta el getter, error en la vista; si falta el setter, el dato no se guarda.

2.4 toString, null, composición, static y final

- **`toString()`** con `@Override`: cómo se muestra el objeto como texto (útil al depurar).
- **`null`**: la variable no apunta a ningún objeto. Llamar a un método sobre null lanza **`NullPointerException`**: compruébalo antes (`if (producto != null)`).
- **Composición**: una clase tiene un atributo que es otro objeto. Es exactamente la relación del ejercicio: **`Moneda`** **tiene una** **`Coleccion`**.

- **`static`**: pertenece a la clase, no a cada objeto. Las clases `XxxData` del ejercicio usan métodos static.
- **`final`**: no puede cambiar tras asignarse (`final double IVA = 0.21;`). En Spring se usa en los atributos inyectados por constructor.

```
public class Alumno {
    private String nombre;
    private Direccion direccion;   // composición: Alumno TIENE una Direccion
}
```

2.5 Herencia

`extends` hereda atributos y métodos de **una sola** clase padre. Debe ser una relación "es un" (un Perro **es un** Animal).

- `super(...)` llama al constructor del padre; `super.metodo()` a un método del padre.
- `@Override` indica que sobrescribes un método heredado.
- **Clase abstracta** (`abstract class`): no se puede instanciar; puede tener métodos abstractos (sin cuerpo) que las hijas deben implementar.
- `sealed ... permits A, B` limita quién hereda; `final` impide heredar; `non-sealed` vuelve a abrir la herencia.

2.6 Interfaces y polimorfismo (muy importante para Spring)

Una **interfaz** es un contrato: dice **qué** métodos existen, no **cómo** se hacen. Una clase la cumple con `implements` (puede implementar varias).

```
public interface MonedaService {          // el contrato
    List<Moneda> listarTodas();
}

public class MonedaServiceImpl implements MonedaService {   // una
implementación
    @Override
    public List<Moneda> listarTodas() { /* ... */ }
}
```

Polimorfismo: una variable del tipo de la interfaz (o del padre) puede apuntar a cualquier objeto que la implemente, y se ejecuta el método del objeto real:

```
MonedaService servicio = new MonedaServiceImpl();   // tipo interfaz, objeto
Impl
```

Los apuntes lo dicen expresamente: "esto será especialmente importante al trabajar con interfaces y Spring". El ejercicio exige que **los controladores dependan solo de la interfaz**: el día que cambies la implementación en memoria por una de base de datos, el controlador no se toca.

Otros elementos de interfaces: métodos `default` (con implementación), métodos `static`, herencia entre interfaces (`interface Mascota extends Animal`) e **interfaces funcionales** (`@FunctionalInterface`, un único método abstracto; permiten lambdas, apartado 3).

Concepto	Sintaxis
Heredar clase	`extends`
Implementar interfaz	`implements`
Heredar interfaz	`extends`
Sobrescribir	`@Override`
Clase abstracta	`abstract class`
Método por defecto	`default`
Herencia restringida / impedida	`sealed` / `final`

2.7 enum [C]

No aparece en los apuntes, pero es la forma natural de un atributo con valores cerrados (estado, dificultad, conservación de una moneda). Spring convierte el texto del formulario (`"MEDIA"`) en la constante (`Dificultad.MEDIA`) automáticamente.

```
public enum Conservacion { SC, EBC, MBC, BC }
// uso: if (moneda.getConservacion() == Conservacion.SC) { ... }   (los enum se comparan con ==)
```

3. Colecciones, genéricos, lambdas, Streams y Optional [A – UT0]

Con datos en memoria, tus servicios **son** listas: listar, buscar, filtrar y ordenar se hace con colecciones y Streams. Es la parte de Java que más vas a escribir en el examen.

3.1 List, Set y Map

Interfaz	Característica	Implementación habitual
`List`	Ordenada, admite repetidos, acceso por posición	`ArrayList`
`Set`	Sin repetidos	`HashSet` (sin orden), `LinkedHashSet` (orden de inserción), `TreeSet` (ordenado)
`Map`	Pares clave → valor	`HashMap`, `LinkedHashMap` (orden de inserción), `TreeMap` (claves ordenadas)
`Queue` / `Deque`	Cola FIFO / doble extremo (cola o pila LIFO)	`LinkedList`, `ArrayDeque` (`Stack` es antigua)

```
List<String> nombres = new ArrayList<>();
nombres.add("Ana");            // añadir
nombres.get(0);                // por posición
nombres.size();                // tamaño
nombres.remove("Ana");         // quitar
nombres.contains("Luis");      // ¿está?
nombres.isEmpty();
nombres.set(0, "Pedro");       // [C] sustituir la posición 0 (lo usamos para
EDITAR)
nombres.removeIf(n -> n.startsWith("A"));   // [C] borrar los que cumplan (lo
usamos para BORRAR)

Map<Integer, String> alumnos = new HashMap<>();
alumnos.put(1, "Ana");
alumnos.get(1);
for (Map.Entry<Integer, String> e : alumnos.entrySet()) {
    System.out.println(e.getKey() + " - " + e.getValue());
}
```

`List.of()` es inmutable: `List.of("a", "b").add("c")` lanza error **en ejecución**. Para los datos del ejercicio, que se dan de alta y se borran, usa `new ArrayList<>()`. [Aviso deducido del ejercicio: es un fallo muy probable.]

3.2 Genéricos y var

`List<Moneda>` indica qué tipo guarda la colección; el compilador impide meter otra cosa. También hay clases genéricas (`class Caja<T>`). `var` infiere el tipo de una **variable local** (`var lista = new ArrayList<String>();`), pero el tipo sigue siendo fijo.

3.3 Ordenar: Comparable y Comparator

- **Comparable**: la clase define su orden natural (`implements Comparable<Alumno>` + `compareTo`). Luego `Collections.sort(lista)` .
- **Comparator**: el criterio va fuera. Es lo más práctico:

```
alumnos.sort(Comparator.comparing(Alumno::getNombre));
alumnos.sort(Comparator.comparing(Alumno::getNombre).reversed());   // [C] descendente
```

3.4 Lambdas e interfaces funcionales

Una lambda es una función escrita en línea: `(parámetros) -> expresión` .

```
n -> n * 2                        // un parámetro (sin paréntesis)
(a, b) -> a + b                   // varios
() -> Math.random()               // ninguno
n -> { int r = n * 2; return r; }   // varias instrucciones: llaves y return
```

Una lambda implementa una interfaz funcional. Las de Java más usadas:

Interfaz	Recibe	Devuelve	Método	Dónde la usas
`Predicate<T>`	T	boolean	`test()`	`filter(...)` , `removeIf(...)`
`Function<T,R>`	T	R	`apply()`	`map(...)`
`Consumer<T>`	T	nada	`accept()`	`forEach(...)`
`Supplier<T>`	nada	T	`get()`	`orElseGet(...)`

Referencias a métodos: `Moneda::getNombre` equivale a `m -> m.getNombre()` ; `System.out::println` equivale a `x -> System.out.println(x)` .

3.5 Stream API

Un Stream procesa los elementos de una colección en cadena **sin modificar la colección original**. Hay operaciones intermedias (devuelven otro Stream) y terminales (dan el resultado y cierran el Stream).

Operación	Tipo	Qué hace
`filter(cond)`	intermedia	Deja pasar los que cumplen la condición
`map(f)`	intermedia	Transforma cada elemento
`sorted()` / `sorted(comparator)`	intermedia	Ordena
`distinct()`	intermedia	Quita repetidos
`limit(n)` / `skip(n)`	intermedia	Primeros n / salta n
`toList()`	terminal	Nueva lista
`forEach(acción)`	terminal	Hace algo con cada uno
`count()`	terminal	Cuenta (`long`)
`findFirst()`	terminal	Primer elemento, como `Optional`
`anyMatch` / `allMatch` / `noneMatch`	terminal	¿Alguno / todos / ninguno cumple?
`reduce(0, Integer::sum)`	terminal	Combina todo en un valor

Patrón que se repite en todo el ejercicio («filtrar, ordenar, recoger»):

```
List<Moneda> resultado = monedas.stream()
        .filter(m -> m.getColeccion().getId().equals(coleccionId))   //
filtro por colección
        .filter(m ->
m.getNombre().toLowerCase().contains(texto.toLowerCase()))  // búsqueda
        .sorted(Comparator.comparing(Moneda::getNombre))
        .toList();

List<Moneda> ultimas = monedas.stream()

.sorted(Comparator.comparing(Moneda::getFechaIncorporacion).reversed())
        .limit(4)                                                          //
últimas 4
        .toList();
```

[C] Para números hay Streams especializados con `sum()`, `average()` y `max()`: `monedas.stream().mapToDouble(Moneda::getValorEstimado).sum()`.

3.6 Optional

`Optional<T>` es una caja que puede contener un valor o estar vacía. Obliga a pensar en el caso «no encontrado». Es lo que devuelve `findFirst()` y lo que deben devolver tus métodos `buscarPorId`.

```
Optional<Moneda> resultado = monedas.stream().filter(m ->
m.getId().equals(id)).findFirst();

if (resultado.isPresent()) { Moneda m = resultado.get(); }   // [A]
resultado.ifPresent(System.out::println);                    // [A]
resultado.isEmpty();                                          // [C] true si
no hay valor
Moneda m = resultado.orElse(null);                            // [C] valor o
null
Moneda m2 = resultado.orElseThrow();                          // [C] valor o
excepción
```

Nunca llames a `.get()` sin comprobar antes: si está vacío lanza `NoSuchElementException`.

4. Aplicaciones web, MVC y Spring Boot [A – UT2 parte 1]

4.1 Código de servidor

Una página estática es HTML fijo. Con código de servidor, el HTML se **genera** cuando llega la petición HTTP del navegador. El código de servidor solo accede a recursos del servidor o de la red, **nunca a los del cliente**, y su comportamiento se configura en archivos como `application.properties`.

Java se compila a **bytecode**, que ejecuta la **JVM**: por eso es multiplataforma. Los fuentes tienen extensión `.java`.

4.2 Qué es Spring Boot

Spring Framework es el gran framework de Java para aplicaciones. **Spring Boot** es un subproyecto que lo simplifica:

- **Configuración por defecto** (sin XML) para Spring MVC, Spring Data JPA, Spring Security, REST…
- **Gestión de dependencias** con **Maven**: tú pides un "starter" y Boot trae las librerías compatibles.
- **Servidor Tomcat embebido**: la aplicación se arranca sola, sin desplegar un WAR (`java -jar miapp.jar`).

- Un modelo de programación de siempre: arranca con `main`.
- Extras para producción: logs, endpoints de salud, métricas.

4.3 Crear el proyecto

Con **Spring Initializr** (https://start.spring.io) o desde IntelliJ (File → New → Project → Spring Boot, que usa ese mismo servicio):

- Project: **Maven** · Language: **Java** · Spring Boot: la versión más reciente · Packaging: **Jar**.
- **Group**: paquete base (ej. `es.ivanlorenzo`) · **Artifact**: nombre del proyecto (ej. `miweb`) · Package name = Group.Artifact.
- Dependencias que citan los apuntes: **Spring Web**, **Spring Boot DevTools**, **Lombok**, **Thymeleaf**, Spring Data JPA + MySQL y Spring Security. Para el ejercicio hacen falta Web, DevTools, Lombok, Thymeleaf y **Validation** (UT4 4). JPA, MySQL y Security **no**: Security, además, pondría un login en todas las páginas.

Se ejecuta con el botón Run sobre la clase principal o con `mvn spring-boot:run`. Si arranca bien verás `Tomcat started on port 8080`. El puerto se cambia con `server.port=8000` en `application.properties`.

4.4 La clase principal

```
@SpringBootApplication
public class MiwebApplication {
    public static void main(String[] args) {
        SpringApplication.run(MiwebApplication.class, args);
    }
}
```

`@SpringBootApplication` activa la configuración automática y [C] el **escaneo de componentes**: Spring busca clases con `@Controller`, `@Service`, `@Configuration`… **en el paquete de esta clase y sus subpaquetes**. Por eso `controller`, `service`, etc. deben colgar del paquete principal (`es.ivanlorenzo.miweb.controller`). Si pones un controlador fuera, Spring no lo encuentra y la URL da error 404.

4.5 Patrón MVC

Separa la aplicación en tres partes:

Parte	Responsabilidad	En nuestro proyecto
Modelo	Los datos: obtenerlos, modificarlos, guardarlos	Clases de `model` + servicios + datos

Parte	Responsabilidad	En nuestro proyecto
Vista	La interfaz de usuario (HTML)	Plantillas Thymeleaf en `templates/`
Controlador	Decide qué hacer con cada petición; habla con el modelo y elige la vista	Clases de `controller`

Flujo: el usuario pide una URL → el controlador pide los datos al modelo → el modelo los devuelve → el controlador elige una vista y le pasa los datos → se devuelve el HTML. Ventaja: puedes cambiar la vista (por ejemplo, una para móvil) sin tocar ni el modelo ni el controlador.

4.6 Spring MVC

Es el framework web de Spring basado en MVC. Usa el patrón **Front Controller**: todas las peticiones llegan a un único servlet, el **DispatcherServlet**, que las reparte al método de controlador cuya ruta coincide y luego renderiza la vista. Tú nunca escribes el DispatcherServlet: lo configura Spring Boot.

Genera respuestas en HTML, JSON o XML con distintos motores de vista (JSP, **Thymeleaf**, Freemarker) y usa el contenedor **IoC** (inversión de control) para la inyección de dependencias (apartado 7).

5. Controladores y Model [A – UT2, UT3 1.4, UT4 1]

Un controlador es una clase que recibe peticiones HTTP y decide la respuesta. Lo habitual es **un controlador por entidad** (`MonedaController` , `ColeccionistaController`), no uno por pantalla.

5.1 @RestController vs @Controller

	`@RestController`	`@Controller`
Qué devuelve el método	**Datos** (texto o JSON) que se envían tal cual	El **nombre de una vista**
`return "inicio";` muestra…	El texto `inicio`	La plantilla `templates/inicio.html`
Uso	APIs REST	Webs con Thymeleaf ← **el ejercicio**

Error típico: poner `@RestController` en una web con Thymeleaf. La página mostrará literalmente la palabra `monedas/lista` en vez del HTML.

5.2 Mapeo de rutas

Anotación	Método HTTP	Uso
`@GetMapping`	GET	Obtener / mostrar (listados, detalle, formularios vacíos)
`@PostMapping`	POST	Enviar datos (guardar un formulario, borrar)
`@PutMapping`	PUT	Actualizar entero (APIs)
`@DeleteMapping`	DELETE	Eliminar (APIs)
`@PatchMapping`	PATCH	Actualizar en parte (APIs)
`@RequestMapping`	Cualquiera	Ruta genérica; a nivel de clase, prefijo común

```
@Controller
@RequestMapping("/admin")            // nivel de clase: prefijo para todos
los métodos
public class AdminController {
    @GetMapping("/panel")             // GET  /admin/panel
    public String panel() { return "panel-admin"; }

    @PostMapping("/crear")            // POST /admin/crear
    public String crear() { return "usuario-creado"; }
}
// Forma antigua equivalente: @RequestMapping(value = "/lista", method =
RequestMethod.GET)
```

Los atajos `@GetMapping`, `@PostMapping`… existen desde Spring 4.3. [C] Un formulario HTML **solo** puede enviar GET o POST: por eso en una web con Thymeleaf el borrado se hace con un `@PostMapping("/{id}/borrar")` y no con `@DeleteMapping`.

5.3 Model: el «maletín» hacia la vista

`Model` es lo que viaja del controlador a la plantilla. El controlador mete datos con `addAttribute(nombre, valor)` y la plantilla los lee con `${nombre}`. Basta con añadir `Model model` como parámetro: Spring te lo pasa.

```
@GetMapping("/hola")
public String hola(Model model) {
    model.addAttribute("nombre", "Lucía");
    model.addAttribute("fecha", LocalDate.now());
    return "hola";                    // templates/hola.html
}
```

```
<p th:text="${nombre}">nombre por defecto</p>
```

El flujo es siempre el mismo: GET /ruta → método del controlador → mete datos en Model → devuelve nombre de vista → Thymeleaf fusiona Model + HTML → al navegador llega **HTML puro** (Thymeleaf no existe en el cliente).

5.4 Datos desde la URL: @PathVariable y @RequestParam

Regla práctica de los apuntes: `@PathVariable` identifica un elemento concreto; `@RequestParam` filtra o busca.

```
// /productos/5   → id = 5
@GetMapping("/productos/{id}")
public String ver(@PathVariable Long id, Model model) { ... }

// /productos/legumbres/7 → varias variables de ruta
@GetMapping("/productos/{categoria}/{id}")
public String ver(@PathVariable("categoria") String categoria,
@PathVariable("id") int id) { ... }

// /productos/buscar?nombre=camisa&precio=30
@GetMapping("/productos/buscar")
public String buscar(@RequestParam("nombre") String nombre,
                     @RequestParam(value = "precio", required = false,
defaultValue = "0") float precio) { ... }
```

- Un `@RequestParam` es **obligatorio por defecto**: si falta, error 400. Para que sea opcional: `required = false` (llega `null`) o `defaultValue`.
- Los parámetros llegan como texto; Spring los **convierte** al tipo del parámetro (`int`, `Long`, `double`, `enum`…). Si no puede (texto en un número), error 400.
- Los campos de un formulario enviado por GET llegan como `@RequestParam`. Con POST también se pueden recoger así.
- [C] Si el nombre del parámetro Java coincide con el de la URL, puedes omitirlo: `@PathVariable Long id`.

5.5 Ámbito de variables en un controlador [A – UT2 p2]

Spring crea **una sola instancia** de cada controlador (singleton) y la comparten todos los usuarios. Una variable local de un método vive solo durante esa petición; un **atributo de instancia** del controlador es **compartido por todos** (el ejemplo del contador de peticiones). Conclusión: no guardes datos de usuario en atributos del controlador.

[C] Por la misma razón, la lista en memoria de un servicio también es compartida: si un usuario da de alta una moneda, los demás la ven. Es justo lo que queremos (una «base de datos» en memoria), aunque se pierde al reiniciar.

6. Thymeleaf [A – UT2 p2, UT3]

Thymeleaf es el motor de plantillas: toma un HTML con atributos `th:` y los datos del Model, y produce HTML normal. Ventajas frente a JSP: no necesita servidor para abrir la plantilla, usa HTML estándar y se integra mejor con Spring Boot y Bootstrap.

6.1 Puesta en marcha

- Dependencia `spring-boot-starter-thymeleaf` (la añade Initializr al marcar Thymeleaf).
- Plantillas en `src/main/resources/templates/`. `return "monedas/lista"` busca `templates/monedas/lista.html`.
- En cada plantilla: `<html xmlns:th="http://www.thymeleaf.org">`.
- `spring.thymeleaf.cache=false` en `application.properties` para ver los cambios sin reiniciar.

6.2 Tipos de expresión

Expresión	Qué es	Ejemplo
`${...}`	Variable del Model	`${moneda.nombre}` (llama a `getNombre()`)
`*{...}`	Atributo del objeto del formulario (`th:object`)	`*{nombre}`
`@{...}`	URL relativa al contextPath	`@{/monedas}`
`#...`	Objetos de utilidad [C, salvo `#fields`]	`#lists.isEmpty(x)`, `#temporals.format(f, 'dd/MM/yyyy')`, `#numbers.formatDecimal(...)`

Expresión	Qué es	Ejemplo
`[[${x}]]` / `[(${x})]`	Variable dentro del texto (escapada / sin escapar)	`<p>Hoy es [[${fecha}]]</p>`

Concatenar: `th:text="'Fecha: ' + ${fecha}"`. [C] Navegación segura: `${moneda.coleccion?.nombre}` no falla si `coleccion` es null.

6.3 Atributos principales

Atributo	Para qué
`th:text`	Sustituye el contenido de la etiqueta (escapa HTML)
`th:each="m : ${monedas}"`	Repite la etiqueta por cada elemento
`th:if` / `th:unless`	Muestra la etiqueta si se cumple / si NO se cumple
`th:switch` + `th:case`	Varios casos; `th:case="*"` es el default
`th:href`, `th:src`, `th:action`	Enlaces, imágenes, destino de formularios (siempre con `@{}`)
`th:value`, `th:selected`, `th:checked`	Valores de inputs sueltos [C]
`th:class`, `th:style`, `th:classappend`	Clases o estilos según condición
`th:object`, `th:field`, `th:errors`	Formularios (apartado 8)
`th:fragment`, `th:insert`, `th:replace`	Fragmentos y layout

```
<ul>
    <li th:each="m : ${monedas}" th:text="${m.nombre}">Nombre</li>
</ul>

<span th:if="${alumno.genero == 'F'}">Femenino</span>
<span th:unless="${alumno.genero == 'F'}">Masculino</span>

<div th:switch="${pedido.estado}">
    <span th:case="'PENDIENTE'">En proceso</span>
    <span th:case="'ENVIADO'">Enviado</span>
    <span th:case="*">Desconocido</span>
</div>
```

6.4 Ternario y operador Elvis

Cuidado: los apuntes de la UT3 mezclan los dos nombres (ver apartado 12).

- **Ternario** `condición ? A : B`: `th:text="${usuario.estatus == 1} ? 'ACTIVO' : 'BLOQUEADO'"`.
- **Elvis** `valor ?: porDefecto`: si el valor es null usa el otro. `th:text="${mensaje} ?: 'Texto por defecto'"`.

6.5 URLs con @{...}

Las rutas con `@{}` empiezan por `/` y Thymeleaf les añade el contextPath, así funcionan en cualquier servidor.

```
<a th:href="@{/monedas/{id}(id=${m.id})}">Ver</a>          <!-- /monedas/5
(variable de ruta) -->
<a th:href="@{/monedas(coleccionId=2)}">Peseta</a>          <!-- /monedas?
coleccionId=2 (parámetro) -->
<a th:href="@{/monedas/{id}/editar(id=${m.id})}">Editar</a>
<link th:href="@{/css/estilos.css}" rel="stylesheet">     <!--
static/css/estilos.css -->
<img th:src="@{/imagenes/{img}(img=${m.imagen})}">         <!-- imagen
dinámica -->
```

Regla: si el nombre entre paréntesis aparece entre llaves en la ruta, sustituye la variable; si no, se añade como `?nombre=valor`. Los recursos estáticos (CSS, JS, imágenes fijas) van en `src/main/resources/static/`. Bootstrap desde CDN se enlaza con HTML normal, sin `@{}`.

6.6 Fragmentos y layout común

Un fragmento es un trozo de HTML reutilizable (cabecera, menú, pie). Se define con `th:fragment` y se incluye con `th:insert` o `th:replace`.

```
<!-- templates/fragments/layout.html -->
<header th:fragment="cabecera"> ... </header>
<footer th:fragment="pie(pagina)">© 2026 – [[${pagina}]]</footer>   <!-- con
parámetro -->

<!-- en cualquier página -->
<div th:insert="~{fragments/layout :: cabecera}"></div>
<footer th:replace="~{fragments/layout :: pie('Inicio')}"></footer>
```

	`th:insert`	`th:replace`
Resultado	Mete el fragmento **dentro** del contenedor	**Sustituye** el contenedor por el fragmento
`<div th:insert>` + `<header>`	`<div><header>…</header></div>`	`<header>…</header>`

[C] Los apuntes escriben `th:insert="fragments/header :: cabecera"`; la forma recomendada en Thymeleaf 3.1 es con `~{...}`: `th:insert="~{fragments/header :: cabecera}"`. La primera aún funciona pero muestra un aviso de obsoleta en consola.

El ejercicio exige «layout común con fragmentos (cabecera, menú y pie) y Bootstrap» [E]: un `layout.html` con fragmentos `head(titulo)`, `cabecera`, `menu`, `mensajes` y `pie` que cada página incluye con `th:replace`. Está completo en la Chuleta.

6.7 Formularios HTML básicos

- `GET`: los datos viajan en la URL (`?v1=0&v2=3`). Para **recuperar** información: búsquedas y filtros.
- `POST`: los datos viajan en el cuerpo de la petición. Para **enviar** información: altas, ediciones, borrados.
- Ninguno es más seguro que el otro.
- En Thymeleaf el destino va en `th:action="@{/ruta}"`; el `name` de cada input es el nombre del parámetro que llega al servidor.

```
<form th:action="@{/monedas}" method="get">
    <input type="text" name="nombre">        <!-- llega como @RequestParam
String nombre -->
    <button type="submit">Buscar</button>
</form>
```

7. Capa de servicio e inyección de dependencias [A – UT4 2]

La capa de servicio es la intermediaria entre los controladores y los datos. Contiene la **lógica de negocio** (buscar, filtrar, calcular, guardar) para que el controlador no trabaje directamente con los datos.

7.1 @Service y el contenedor de beans

Spring tiene un **contenedor** que crea, inicializa y gestiona los objetos de la aplicación. A esos objetos los llama **beans**. Al poner `@Service` en una clase:

- Spring la detecta (escaneo de componentes) y crea **una única instancia** (singleton por defecto).
- Esa instancia se puede **inyectar** en controladores u otros servicios.
- **Tú nunca haces `new MonedaServiceImpl()`**: lo hace Spring.

Otras anotaciones que crean beans [C]: `@Controller` (controladores), `@Repository` (acceso a datos, lo verás con JPA), `@Component` (genérica) y `@Configuration` (clases de configuración). Las clases de `model`, `data` y `util` **no** llevan anotación: no son beans.

7.2 Servicio con interfaz (lo que exige el ejercicio)

```
public interface ProductoService {            // contrato
    Producto buscarPorId(int id);
}

@Service                                       // la anotación va en la
IMPLEMENTACIÓN
public class ProductoServiceImpl implements ProductoService {
    @Override
    public Producto buscarPorId(int id) { ... }
}
```

El controlador declara un atributo del tipo **interfaz** y Spring le inyecta la implementación que encuentre. Código «flexible y desacoplado»: si mañana existe `ProductoServiceJpa`, el controlador no cambia.

[C] Si hay **dos** clases `@Service` que implementan la misma interfaz, Spring no sabe cuál inyectar y la aplicación no arranca (`NoUniqueBeanDefinitionException`). Si no hay **ninguna** (te falta `@Service`), tampoco arranca: `required a bean of type '...MonedaService' that could not be found`.

7.3 Tres formas de inyectar con @Autowired

Forma	Cómo	Valoración de los apuntes
Constructor	Atributo `private final` + constructor que lo recibe	Más clara, permite `final`, ideal para pruebas. **Recomendada**
Atributo	`@Autowired private ProductoService servicio;`	Más rápida de escribir, menos clara, no permite `final`
Setter	`@Autowired` sobre un `setXxx(...)`	Para dependencias opcionales; poco usada

```
@Controller
@RequestMapping("/productos")
public class ProductoController {

    private final ProductoService productoService;

    @Autowired     // [C] opcional si la clase tiene UN único constructor
    public ProductoController(ProductoService productoService) {
        this.productoService = productoService;
    }
}
```

[C] Con un solo constructor, Spring lo usa automáticamente para inyectar, aunque no pongas `@Autowired`. Los apuntes lo ponen; las dos formas son correctas.

7.4 Servicio en memoria (lo deduce el ejercicio) [E]

El enunciado pide que cada servicio sea una interfaz con una implementación **en memoria** y que los datos iniciales vengan de clases `PiezaData`, `ColeccionData` y `ColeccionistaData`, **cargadas desde el constructor del servicio**. Eso significa:

```
@Service
public class MonedaServiceImpl implements MonedaService {

    private final List<Moneda> monedas;   // la «base de datos»
    private Long siguienteId;              // para dar ids a las altas

    public MonedaServiceImpl(ColeccionService coleccionService) {   //
inyección en un servicio
        this.monedas = MonedaData.getMonedas(coleccionService.listarTodas());
        this.siguienteId =
monedas.stream().mapToLong(Moneda::getId).max().orElse(0L) + 1;
    }
    // listarTodas, buscarPorId, guardar, eliminar... con Streams (apartado
3)
}
```

Como el servicio es singleton, la lista vive mientras la aplicación esté arrancada. Al reiniciar vuelve a los datos iniciales. Está resuelto completo en el documento 3.

8. Formularios, validación y redirecciones [A – UT3 7, UT4 3–5]

8.1 Clase modelo y Lombok

Una clase modelo representa los datos que viajan entre controlador y vista: atributos, getters/setters y, opcionalmente, constructores, `toString()` y anotaciones de validación. Va en el paquete `model`.

Lombok genera ese código repetitivo con anotaciones:

Anotación	Genera
`@Getter` / `@Setter`	Getters / setters de todos los atributos
`@ToString`	`toString()`
`@EqualsAndHashCode`	`equals()` y `hashCode()`
`@NoArgsConstructor`	Constructor vacío
`@AllArgsConstructor`	Constructor con todos los atributos (en el orden en que están declarados)
`@RequiredArgsConstructor`	Constructor con los atributos `final` o `@NonNull`
`@Data`	`@Getter` + `@Setter` + `@ToString` + `@EqualsAndHashCode` + `@RequiredArgsConstructor`
`@Builder`	Patrón builder: `Clase.builder().campo(v).build()`
`@Value`	Clase inmutable: campos `private final`, solo getters

```
@Data
@NoArgsConstructor        // [C] imprescindible para los formularios
@AllArgsConstructor       // [C] cómodo para crear los datos iniciales
public class Usuario {
    private String nombre;
    private String email;
    private boolean activo;
}
```

[C] **Combinación recomendada: `@Data @NoArgsConstructor @AllArgsConstructor`.** Los apuntes usan `new Usuario("Iván", "ivan@...")` con una clase que solo tiene `@Data`: eso no compila, porque `@Data` no genera ese constructor (ver apartado 12).

8.2 Formulario enlazado a un objeto

El patrón son siempre **dos métodos**: un GET que muestra el formulario con un objeto (vacío o relleno) y un POST que lo recibe.

```
@GetMapping("/nuevo")
public String mostrarFormulario(Model model) {
    model.addAttribute("usuario", new Usuario());     // objeto vacío
    return "form-usuario";
}

@PostMapping("/guardar")
public String guardarUsuario(@ModelAttribute Usuario usuario) {
    // usuario ya viene relleno con lo que escribió el usuario
    return "resultado";
}
```

```
<form th:action="@{/usuarios/guardar}" th:object="${usuario}" method="post">
    <input type="text" th:field="*{nombre}">
    <input type="email" th:field="*{email}">
    <input type="checkbox" th:field="*{activo}">
    <button type="submit">Enviar</button>
</form>
```

- `th:object="${usuario}"` vincula el formulario al objeto que mandó el controlador. El nombre debe coincidir con el `addAttribute`.
- `th:field="*{nombre}"` hace el **data binding**: rellena el input con el valor actual y genera `id` y `name` (`name="nombre"`).
- `@ModelAttribute Usuario usuario` en el POST: Spring crea un `Usuario` (con el constructor vacío) y llama a `setNombre(...)`, `setEmail(...)` con los valores recibidos. Te ahorra un `@RequestParam` por campo.

@ModelAttribute sobre un método (no un parámetro): se ejecuta **antes de cada handler** del controlador y deja ese dato en el Model de todas sus vistas. Perfecto para listas de un `<select>`:

```
@ModelAttribute("colecciones")
public List<Coleccion> cargarColecciones() {
    return coleccionService.listarTodas();
}
```

8.3 Validación

Dependencia `spring-boot-starter-validation`. Al hacer el binding pueden aparecer errores de formato (fecha), de conversión (texto en un `double`) u obligatorios vacíos.

Anotación	Comprueba	Tipo habitual
`@NotNull`	No null (admite `""`)	Cualquiera: `Integer`, `LocalDate`, objetos
`@NotEmpty`	No null ni vacío	`String`, listas
`@NotBlank`	No null, ni vacío, ni solo espacios	`String` (**la buena para textos**)
`@Size(min=, max=)`	Longitud	`String`
`@Email`	Formato de email (básico)	`String`
`@Min(v)` / `@Max(v)`	Rango numérico	`Integer`, `Long`
`@Positive` / `@PositiveOrZero`	> 0 / ≥ 0	Números
`@Negative` / `@NegativeOrZero`	< 0 / ≤ 0	Números
`@Pattern(regexp=)`	Expresión regular	`String` (DNI, teléfono, CP)
`@Past` / `@PastOrPresent`	Fecha anterior / ≤ hoy	`LocalDate`
`@Future` / `@FutureOrPresent`	Fecha posterior / ≥ hoy	`LocalDate`
`@AssertTrue` / `@AssertFalse`	Boolean true / false	`boolean`
`@DecimalMin("0.5")` / `@DecimalMax` [C]	Rango con decimales	`Double`, `BigDecimal`

Cada una admite `message = "..."` con el texto del error.

Importante [C]: salvo `@NotNull` / `@NotBlank` / `@NotEmpty`, las demás **dan por válido el null**. Un `@Min(1) Integer anio` vacío pasa la validación: si es obligatorio, añade también `@NotNull`.

En el controlador:

```
@PostMapping("/usuarios/guardar")
public String guardar(@Valid @ModelAttribute Usuario usuario,
                      BindingResult resultado,          // ¡JUSTO DESPUÉS del
objeto validado!
                      Model model) {
    if (resultado.hasErrors()) {
        return "formulario-usuario";   // vuelve al formulario con los
errores
    }
    service.guardar(usuario);
    return "redirect:/usuarios/lista";
}
```

- `@Valid` pide a Spring que valide el objeto según sus anotaciones.
- `BindingResult` guarda los errores. **Si no va inmediatamente detrás del objeto, Spring lanza una excepción (error 400) en vez de dejarte gestionarlos.**
- [C] Errores que las anotaciones no pueden comprobar (email repetido, colección inexistente): `resultado.rejectValue("email", "email.duplicado", "Ya existe ese email");` y aparecen en la vista igual que los demás.

Mostrar los errores en Thymeleaf:

```
<!-- Por campo, junto al input -->
<input type="text" th:field="*{nombre}">
<p th:if="${#fields.hasErrors('nombre')}" th:errors="*{nombre}"></p>

<!-- Todos juntos, al principio del formulario -->
<div th:if="${#fields.hasGlobalErrors() or #fields.hasErrors()}">
    <ul><li th:each="err : ${#fields.errors('*')}" th:text="${err}"></li>
</ul>
</div>
```

8.4 Fechas en formularios [C, imprescindible para el ejercicio]

El ejercicio pide una validación de fecha y atributos de fecha (incorporación, alta). Usa `java.time.LocalDate` y la anotación `@DateTimeFormat(iso = DateTimeFormat.ISO.DATE)`: un `<input type="date">` envía `2025-03-14` y, sin esa anotación, Spring no sabe convertirlo y da error de conversión.

```
@NotNull(message = "La fecha es obligatoria")
@PastOrPresent(message = "No puede ser futura")
@DateTimeFormat(iso = DateTimeFormat.ISO.DATE)
private LocalDate fechaIncorporacion;
```

Para mostrarla: `th:text="${#temporals.format(m.fechaIncorporacion, 'dd/MM/yyyy')}"`. Crear fechas en Java: `LocalDate.of(2024, 3, 20)`, `LocalDate.now()`.

8.5 Redirect y forward

Tras un POST correcto no se devuelve una vista: se **redirige**. Si devolvieras la vista, al recargar (F5) el navegador reenviaría el formulario y crearía un duplicado.

`return "redirect:/usuarios/lista";` envía un **HTTP 302**: el navegador hace un **GET nuevo** a esa URL. Consecuencia: **el Model se pierde** (es de una sola petición). Para mandar un mensaje de confirmación se usan los **atributos flash** de `RedirectAttributes`, que se guardan en sesión y desaparecen tras la redirección:

```
@PostMapping("/usuarios/guardar")
public String guardar(@Valid @ModelAttribute Usuario usuario, BindingResult
resultado,
                      RedirectAttributes redirectAttrs) {
    if (resultado.hasErrors()) {
        return "formulario-usuario";        // SIN redirect: si no, se
pierden los errores
    }
    service.guardar(usuario);
    redirectAttrs.addFlashAttribute("mensaje", "Usuario guardado
correctamente");
    return "redirect:/usuarios/lista";
}
```

```
<p th:if="${mensaje}" th:text="${mensaje}"></p>   <!-- en la página de
destino -->
```

También se pueden pasar parámetros en la URL: `"redirect:/usuarios/lista?exito=true"`.

`forward:/ruta` reenvía la petición **dentro** del servidor: el navegador no se entera y la URL no cambia.

	`redirect:`	`forward:`
¿Nueva petición del cliente?	Sí	No
¿Cambia la URL?	Sí	No
¿Se mantiene el Model?	No (salvo flash attributes)	Sí
Uso típico	Después de un POST	Reutilizar lógica en el servidor

El ejercicio exige redirección con mensaje (`RedirectAttributes`) tras **cada** alta, edición y borrado [E].

9. Subida de imágenes y @Value [A – UT4 6]

El ejercicio pide alta y edición de piezas **con imagen**, así que este apartado entra seguro.

9.1 Configuración

```
# application.properties
spring.servlet.multipart.max-file-size=5MB
spring.servlet.multipart.max-request-size=5MB
# Carpeta donde se guardan (propiedad inventada por nosotros)
file.upload-dir=uploads
```

En Linux o Mac la carpeta necesita permisos de escritura (`chmod -R 755`).

9.2 El formulario

```
<form th:action="@{/imagen/subir}" method="post" enctype="multipart/form-
data">
    <input type="file" name="archivoImagen">
    <button type="submit">Subir</button>
</form>
```

- **`enctype="multipart/form-data"` es obligatorio**: sin él, el fichero no viaja.
- El `name` del input de fichero **no debe coincidir** con el atributo del modelo (`imagen`). El modelo guarda un `String` con el **nombre** del fichero, no el fichero; si coincidieran, Spring intentaría meter un fichero en un String y fallaría.

9.3 El controlador

```
@Value("${file.upload-dir}")
private String uploadDir;

@PostMapping("/subir")
public String subirImagen(@RequestParam("archivoImagen") MultipartFile
archivo) {
    try {
        Path path = Paths.get(uploadDir, archivo.getOriginalFilename());
        Files.createDirectories(path.getParent());   // crea la carpeta si no
existe
        Files.write(path, archivo.getBytes());        // guarda los bytes
        return "redirect:/ruta";
    } catch (IOException e) {
        return "redirect:/error";
    }
}
```

- `MultipartFile` es el fichero recibido: `getOriginalFilename()`, `getBytes()`, `isEmpty()`, `getContentType()` [C].
- `try/catch (IOException e)` [C]: escribir en disco puede fallar, y Java **obliga** a capturar o declarar `IOException` (excepción comprobada).

9.4 ¿Dónde guardar?

No en `resources/static`: esa carpeta se empaqueta dentro del JAR, lo que se sube en ejecución no persiste, puede no tener permisos de escritura y no escala. Los apuntes recomiendan una **carpeta externa** configurable (o almacenamiento en la nube, como AWS S3).

9.5 Lo que los apuntes no explican y el ejercicio necesita [C]

1. **Mostrar las imágenes subidas.** Si guardas en `uploads/`, fuera de `static`, Spring no las sirve por sí solo: `<img th:src="@{/imagenes/...}">` daría 404. Hay que registrar esa carpeta en una clase de **configuración** (`WebMvcConfigurer.addResourceHandlers`). Esto explica el paquete `config` que pide el enunciado.
2. **Nombres repetidos.** Dos usuarios que suben `foto.jpg` se pisarían. Se añade un prefijo único con `UUID.randomUUID()`. Esta lógica encaja en el paquete `util` del enunciado.

3. **Edición sin nueva imagen.** Si el usuario no elige fichero, `archivo.isEmpty()` es true y hay que conservar la imagen anterior (un `<input type="hidden" th:field="*{imagen}">` la mantiene en el formulario).

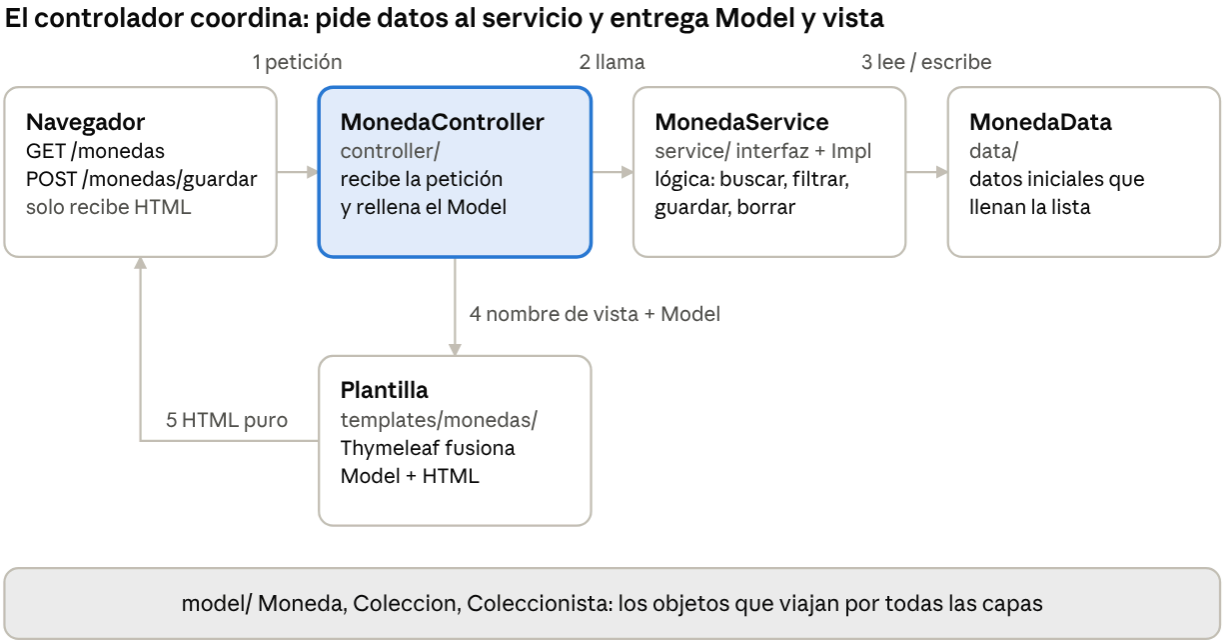
El código completo de `WebConfig` e `ImagenUtil` está en la Chuleta (apartado de imágenes).

9.6 @Value

`@Value("${nombre.propiedad}")` inyecta en un atributo el valor de una propiedad de `application.properties`. Solo funciona dentro de beans (`@Controller`, `@Service`, `@Configuration`...). Si la propiedad no existe, la aplicación no arranca (`Could not resolve placeholder`).

10. Cómo encaja todo [A + E]

Cada capa habla solo con la de al lado; el controlador es el centro. Este es el recorrido de cualquier petición del ejercicio:



Recorrido de una petición por las capas · 5 pasos

El navegador nunca ve Java: recibe HTML. El controlador no toca la lista directamente: se lo pide al servicio a través de su interfaz. Los objetos de `model/` viajan por todas las capas.

10.1 Estructura de carpetas que exige el ejercicio

```
src/main/java/es/daw/appcoleccion/
├── AppcoleccionApplication.java     ← @SpringBootApplication + main
├── model/        Moneda, Coleccion, Coleccionista            (sin anotación
de Spring)
├── service/      MonedaService (interfaz) + MonedaServiceImpl (@Service)...
├── controller/   MonedaController, ColeccionController...     (@Controller)
├── data/         MonedaData, ColeccionData, ColeccionistaData  (métodos
static)
├── util/         ImagenUtil                                    (métodos
static)
└── config/       WebConfig
(@Configuration)
src/main/resources/
├── application.properties
├── static/       css/, imagenes/   (recursos fijos)
└── templates/    fragments/layout.html, index.html, monedas/, colecciones/,
coleccionistas/
```

10.2 Quién conoce a quién

Clase	Anotación	La crea	Recibe por constructor
`MonedaController`	`@Controller`	Spring	`MonedaService`, `ColeccionService` (interfaces)
`MonedaServiceImpl`	`@Service`	Spring	`ColeccionService` (para enlazar cada moneda con su colección)
`MonedaData`	ninguna	nadie (static)	La llama el constructor del servicio
`Moneda`	ninguna	tú (`new`) o Spring al recibir un formulario	Nada
`WebConfig`	`@Configuration`	Spring	Lee `file.upload-dir` con `@Value`

10.3 El ciclo de un formulario de alta

1. `GET /monedas/nueva` → el controlador mete una `new Moneda()` en el Model → vista `monedas/formulario`.
2. El usuario rellena y envía → `POST /monedas/guardar` con `enctype=multipart/form-data`.
3. Spring crea una `Moneda`, llama a sus setters con los campos (`@ModelAttribute`) y valida (`@Valid`). Los errores van a `BindingResult`.
4. Con errores → `return "monedas/formulario"` (sin redirect) y Thymeleaf pinta los mensajes.
5. Sin errores → se guarda la imagen, `monedaService.guardar(moneda)`, mensaje flash y `redirect:/monedas/{id}`.
6. El navegador hace un GET nuevo y la página de destino muestra el mensaje.

11. Complementario: JPA y base de datos [C]

Para este examen no hace falta, salvo que el profesor diga lo contrario. El enunciado de la Hoja04 dice expresamente que los datos estarán en memoria y que «en la siguiente unidad» pasaréis a base de datos con relaciones y herencia. Lo incluyo como resumen porque lo pediste y porque verás que el resto del código apenas cambia.

11.1 Qué cambia

Ahora (en memoria)	Con JPA
`List<Moneda>` dentro de `MonedaServiceImpl`	Tabla `moneda` en MySQL
Clases `XxxData` con datos iniciales	`data.sql` o inserts
`siguienteId++`	`@GeneratedValue` (lo genera la BD)
Streams para buscar	Métodos de `JpaRepository`
`MonedaController`	**Igual** (depende de la interfaz)

11.2 Configuración

Dependencias: **Spring Data JPA** y **MySQL Driver**.

```
spring.datasource.url=jdbc:mysql://localhost:3306/club
spring.datasource.username=root
spring.datasource.password=
# Crea/actualiza las tablas a partir de las entidades
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

11.3 Entidad, relación y repositorio

```
@Entity
@Data @NoArgsConstructor @AllArgsConstructor
public class Moneda {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;

    @ManyToOne                          // muchas monedas → una colección
    @JoinColumn(name = "coleccion_id")
    private Coleccion coleccion;
}

// Interfaz vacía: Spring genera la implementación
public interface MonedaRepository extends JpaRepository<Moneda, Long> {
    List<Moneda> findByNombreContainingIgnoreCase(String texto);   //
consulta derivada del nombre
    List<Moneda> findByColeccionId(Long coleccionId);
}
```

Métodos que trae `JpaRepository` sin escribir nada: `findAll()`, `findById(id)` (devuelve `Optional`), `save(obj)` (alta si el id es null, si no edición), `deleteById(id)`, `existsById(id)`, `count()`. Consultas a medida: `@Query("SELECT m FROM Moneda m WHERE m.valorEstimado > :min")`.

El servicio pasaría a usar el repositorio en vez de la lista: `return monedaRepository.findAll();`. Relaciones: `@ManyToOne`, `@OneToMany(mappedBy = ...)`, `@ManyToMany`, `@OneToOne`. Herencia: `@Inheritance`.

12. Erratas y contradicciones de los apuntes

Las he encontrado al revisar el material. Ninguna invalida los apuntes, pero conviene saberlas para no copiarlas en el examen.

Dónde	Qué dice	Lo correcto
UT3 3.1 «Operador Elvis»	Llama Elvis a `${cond} ? 'ACTIVO' : 'BLOQUEADO'`	Eso es el **ternario**. Elvis es `${valor} ?: 'por defecto'`, como aparece bien en la UT3 1.3
UT3 6.1	«GET: el método más usado, pero no se usa en formularios»	La propia UT3 6.4 pone un ejemplo de formulario GET. GET sí se usa en formularios de **búsqueda y filtro**; POST para enviar datos
UT3 7 (`UsuarioController`)	`new Usuario("Iván", "ivan@...")` con una clase que solo tiene `@Data`	No compila: `@Data` no crea ese constructor. Hace falta `@AllArgsConstructor` (y entonces con los 3 atributos) o un constructor escrito a mano
UT2 p1 6.2 (MVC)	«devuelve una vista (página php)» y «frameworks en PHP»	Texto heredado de PHP. En Spring la vista es una plantilla Thymeleaf
UT4 6 (imágenes)	Formulario con `action="imagen/subir"` y controlador con `@PostMapping("/subir")`	Las rutas no coinciden. Deben ser iguales y conviene usar `th:action="@{/subir}"`
UT4 6 (@Value)	«declaramos un atributo privado en el constructor con @Value»	Es un atributo de la **clase** (fuera del constructor)
UT4 4 (tabla de validación)	`@Max(100) private int porcentaje` (sin decimales)	Correcto para enteros. Para `Double` es más seguro `@DecimalMax("100.0")`
UT3 5 (fragments)	`th:insert="fragments/header :: cabecera"`	Funciona, pero en Thymeleaf 3.1 se recomienda `~{fragments/header :: cabecera}`
UT2 p2	Plantilla `hola.html` con un comentario `<!-- ... -` sin cerrar	Errata de formato: un comentario HTML se cierra con `-->`

Dónde	Qué dice	Lo correcto
Hoja04	Pide el paquete `data` y además `util` y `config` sin decir qué contienen	Deducido: `config` para servir las imágenes subidas y `util` para la lógica de guardado de ficheros (apartado 9.5)

Si el profesor usa en clase otra convención distinta de las correcciones de esta tabla, haz caso al profesor.